07 - Dynamic Programming

Algorithms & Complexity

Eduard Torres Montiel - eduard.torres@udl.cat

Logic & Optimization Group, University of Lleida

22/05/2024

Computing Fibonacci numbers

Probably, you are familiar with *Fibonacci numbers*. These numbers can be built using the following mathematical function:

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases} \quad (1)$$

Probably, you are also familiar with the algorithm that computes any Fibonacci number:

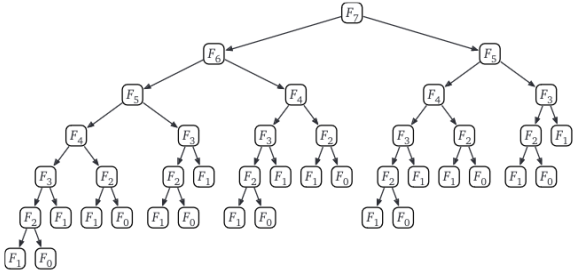
Algorithm 1 The `Fibonacci` recursive algorithm.

procedure RecFibo(n)
  if $n = 0$ **then return** 0
  else if $n = 1$ **then return** 1
  elsereturn $RecFibo(n-1) + RecFibo(n-2)$

Notice how this algorithm is indeed using a **backtracking** scheme (although in this case we never *prune*).

Complexity of this algorithm?

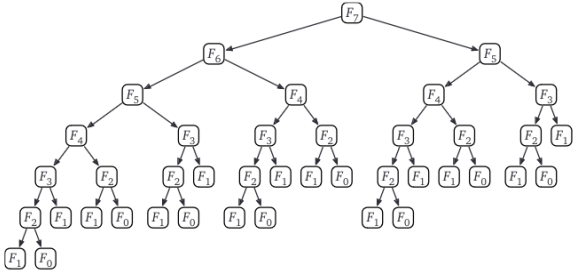
This is the recursion tree for computing $F(7)$:



Source: *Algorithms*, by Jeff Erickson

Each arrow represents a recursive call. Therefore, the complexity of this algorithm is:

This is the recursion tree for computing $F(7)$:



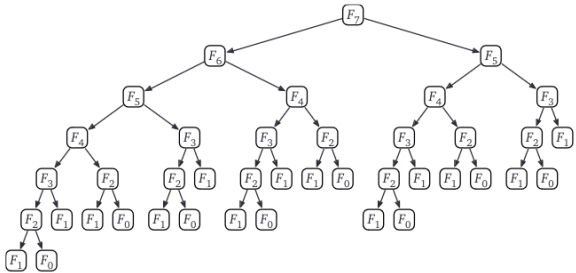
Source: *Algorithms*, by Jeff Erickson

Each arrow represents a recursive call. Therefore, the complexity of this algorithm is:

$O(2^n)$. With more advanced techniques we can demonstrate that its complexity is $\Omega(\phi^n)$, where $\phi = 1.61803...$ (i.e. the *golden ratio*).

Improving the Fibonacci recursive algorithm

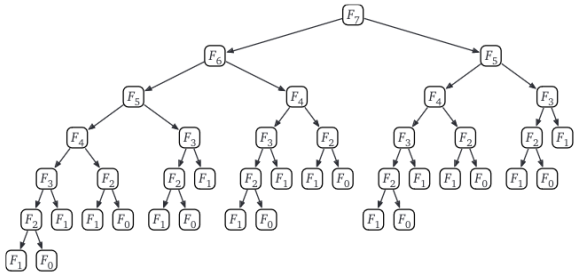
Let's analyze the recursion tree again. Can you see some **patterns**?



Source: *Algorithms*, by Jeff Erickson

Improving the Fibonacci recursive algorithm

Let's analyze the recursion tree again. Can you see some **patterns**?



Source: *Algorithms*, by Jeff Erickson

We are computing **the same** $F(2)$ 8 times, $F(3)$ 5 times, $F(4)$ 4 times, $F(5)$ 2 times!

Indeed, all these computations should be done **only once**!

Introducing *memoization*

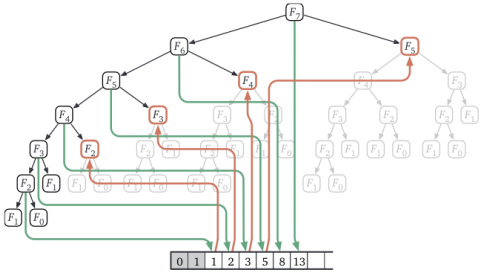
Memoization (without the "R") is a technique that writes down the intermediate results of our recursive calls and looking them up again if we need them later.

Let's see how to create a version of the Fibonacci algorithm that applies *memoization*:

Algorithm 2 The `Fibonacci` algorithm with memoization.

```
procedure MemFibo(n)
    if n = 0 then return 0
    else if n = 1 then return 1
    else
        if F[n] is undefined then
            F[n] ← MemFibo(n − 1) + MemFibo(n − 2)
        return F[n]
```

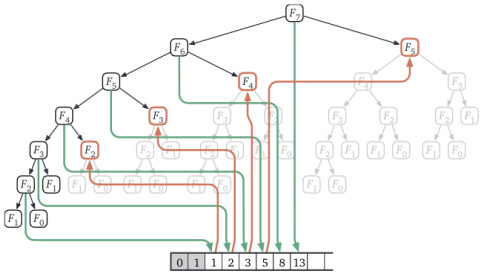
Now, its recursion tree is the following. Downward green arrows indicate writing into the *memoization array*, upward red arrows indicate reading from the *memoization array*.



Source: *Algorithms*, by Jeff Erickson

How many operations are we doing now? (i.e., complexity of the algorithm?)

Now, its recursion tree is the following. Downward green arrows indicate writing into the *memoization array*, upward red arrows indicate reading from the *memoization array*.



Source: *Algorithms*, by Jeff Erickson

How many operations are we doing now? (i.e., complexity of the algorithm?)
$O(n)$, an *exponential* improvement over the naive recursive algorithm!
Notice however that this improvement **is not free**. We are consuming more **space** (in this case memory) and, sometimes, this increment on the memory consumption can be substantial (in some problems we may run out of memory for larger n).

Iterative *memoized* algorithm

Once we see how the array $F[]$ is filled, we can replace the memoized recurrence with a simple `for` loop that *internally* fills the array in that order, instead of relying on a more complicated recursive algorithm:

Algorithm 3 The iterative `Fibonacci` algorithm with memoization.

procedure IterFibo(n)
- $F[0] \leftarrow 0$
- $F[1] \leftarrow 1$
- **for** $i \leftarrow 2$ to n **do**
  - $F[i] \leftarrow F[i-1] + F[i-2]$
- **return** $F[n]$

Now the time analysis is immediate. `IterFibo` uses $O(n)$ additions and stores $O(n)$ integers to F.

This is our first explicit **dynamic programming** algorithm!

Dynamic Programming

In a nutshell, dynamic programming is **recursion without repetition**. Dynamic programming algorithms store the solutions of intermediate subproblems, often *but not always* in some kind of array or table (map, hash, dictionary…).

Sometimes we make the mistake on focusing on the table instead of **finding a correct recurrence**.

So, we say that **Dynamic Programming is not about filling in tables. It's about smart recursion!**

Developing Dynamic Programming algorithms

Formulate the problem recursively. Write down a recursive formula or algorithm for the whole problem in terms of the answers to smaller subproblems.

1. **Specification**: Describe the problem that you want to solve recursively. Not *how*, but *what*.
2. **Solution**: Give a clear recursive formula or algorithm for the whole problem in terms of the answers to smaller instances of **exactly** the same problem.

Build solutions to your recurrence from the bottom up.

1. **Identify the subproblems**. What are all the different ways your recursive algorithm can call itself, starting with some initial input?
2. **Choose a memoization data structure**. Find a data structure that can store the solution to **every** subproblem you identified.
3. **Identify dependencies**. Except for the base cases, every subproblem depends on other subproblems — which ones?
4. **Find a good evaluation order**. Order the subproblems so that each one comes *after* the subproblems it depends on. You should consider the base cases first, then the subproblems that depends only on base cases, and so on.
5. **Analyze space and running time**. The number of distinct subproblems determines the space complexity of your memoized algorithm. To compute the total running time, add up the running times of all possible subproblems.
6. **Write down the algorithm**. You know what order to consider the subproblems, and you know how to solve each subproblem. So do that!

Bibliography

- Jeff Erickson. *Algorithms*. `http://jeffe.cs.illinois.edu/teaching/algorithms/`. Chapter 3.
- Quentin Santos. *Dynamic Programming is not Black Magic*. `https://qsantos.fr/2024/01/04/dynamic-programming-is-not-black-magic/`. Accessed May 2024.